Rewriting the Sigma Calculus

Anonymous author
Anonymous affiliation

Abstract

Choosing a representation of variable bindings and substitutions is a central challenge in mechanizing programming language metatheory. De Bruijn indices provide a canonical representation, but developments based on them require numerous lemmas about substitution composition and interaction with binders, leading to significant proof overhead.

The σ-calculus treats parallel substitutions as first-class objects equipped with primitive operations and a terminating and confluent rewrite system that normalizes substitution expressions. While previous work has exploited these rewrite rules to implement decision procedures, the resulting equalities remain propositional.

Recent support for user-defined rewrite rules in mainstream proof assistants enables the promotion of propositional equalities to definitional ones. To retain logical consistency the set of equational rules must be confluent and supported by proofs. However, obtaining these proofs requires modeling substitutions as functions, which in turn leads to violations of the syntactic restrictions on rewrite rules.

In the context of the proof assistant Agda, we embed the rules of the σ-calculus such that they satisfy the criteria for rewrite rules while retaining their provability. This approach yields built-in normalization of substitutions while preserving logical consistency and full computational behavior.

Our development is reusable: from an Autosubst specification, we automatically generate the required σ-calculus infrastructure and rewrite rules. To our knowledge, this is the first safe integration of the full σ-calculus by definitional rewriting in a proof assistant.

2012 ACM Subject Classification Theory of computation → Equational logic and rewriting

Keywords and phrases Explicit Substitutions, Rewrite Systems, Agda

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Effectively handling variable bindings and substitutions remains a central challenge when mechanizing programming language metatheory. Compared to pen-and-paper proofs, binders in proof assistants are a notorious source of uninteresting technicalities and significant boilerplate. Since the beginning of proof assistants, many representations have been proposed, including locally nameless binders, nominal syntax, higher-order abstract syntax, and de Bruijn indices. However, choosing the right representation is difficult. While de Bruijn syntax provides a promising representation, developments based on it still include substantial proof overhead by requiring numerous lemmas to manage substitution composition and interactions with binders.

On paper, substitution is often an operation that replaces just a single variable. However, in proof assistants we commonly use parallel substitutions, which map variables to terms and replace multiple variables simultaneously when a term is instantiated with them.

1.1 The Sigma Calculus in Autosubst

The σ-calculus [1] treats parallel substitutions as first-class objects equipped with primitive operations and a terminating and confluent [28] rewrite system that normalizes substitution expressions. The σ_{SP} [13] variant complements the original to a complete [25] procedure. We refer to the SP variant simply as the σ-calculus throughout this work.

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

The de Bruijn algebra, where parallel substitutions are modeled as functions, constitutes a semantic model of the σ-calculus. With a semantic model for the substitution operations, the σ-calculus rules become provable, and therefore qualify for equational reasoning in proof assistants [26]. While previous work has exploited these equations to implement automated normalization procedures, the resulting equalities remain propositional. Autosubst [26] provided the first normalization procedure using the σ-calculus rules for unscoped syntax definitions inside the Rocq proof assistant. To normalize terms with parallel substitutions one needs to apply a tactic inside a proof goal.

The successor Autosubst 2 [29] is now regarded as the gold standard for libraries that handle substitution in proof assistants. Autosubst 2 supports intrinsically scoped syntax where the syntax data type is indexed over the number of free variables and variable usage is restricted to the variables in scope by construction. Furthermore, Autosubst 2 generates proofs, for the σ-calculus rules that follow by induction over the syntax, from higher-order abstract syntax descriptions. It also enables reasoning about syntaxes with multiple, mutually dependent, sorts by packaging parallel substitutions over different sorts into vectors.

In particular, Autosubst 2, in contrast to the first version, includes first-class support for renamings, a special case of substitutions that map variables to variables. Independently of first-class support in the equational theory, renamings are required in proof assistants to model substitutions in a structurally recursive manner that the termination checker accepts [3]. The creators of Autosubst 2 extended the equational theory of the σ-calculus with additional rules that handle renamings. The resulting system is not complete anymore, but the authors conjecture that the extended system preserves confluence [28].

1.2 Promoting Propositional to Definitional Equalities with Rewriting

Recent support for user-defined rewrite rules in mainstream proof assistants, such as Agda [10, 12] and Rocq [18], enables the promotion of propositional equalities to definitional ones. To retain logical consistency, the set of equational rules must be confluent and supported by proofs [12]. Agda and other proof assistants perform higher-order pattern unification [2] to detect if a registered rewrite rule can be applied to a stuck term (one that does not reduce further, given the built-in reduction rules). To do so, Agda requires that the left-hand side is a stuck term on a defined symbol.

To model parallel substitution in proof assistants, we have essentially two choices: data types or functions. When we model substitution as functions, some primitive operations of the σ-calculus generate stuck lambdas, thereby violating the syntactic restriction that the left-hand side must be a stuck symbol and not a lambda. Others tried to model substitution with constructors, but then we lack the computational behavior of substitutions required to instantiate the full rule set of the σ-calculus with proofs, and in turn lose preservation of logical consistency. Modeling substitutions as vectors does not constitute a complete semantic model of the σ-calculus.

If we wish to rewrite all σ-calculus rules, we require more control over the reduction behavior of symbols. Fortunately, Agda supports opaque definitions [15], that allow us to selectively block and later unfold the implementation of function definitions. With opaque definitions we can have both: we treat function definitions as stuck symbols, such that we can apply rewrite rules, but also unfold their definitions to prove the rules using their computational behavior.

1.3 Content of this Paper

Contributions

We show how to rewrite the σ-calculus rules, with support for first-class renamings, in a multi-sorted and scoped syntax setting, in Agda. We do so while preserving the logical consistency of the system.

As a side effect we support the claim that the extension of the σ-calculus rules with first-class renamings is confluent, via Agda's built-in confluence checker. We also show that this approach translates to any scoped syntax by implementing a code generation script for higher-order abstract syntax descriptions.

Structure

Section 2 introduces the required background in Agda and for the σ-calculus. Section 3 demonstrates how to register the σ-calculus rules exemplified by an instantiation for System F. We discuss ongoing and future work in Section 4, related work in Section 5 and conclude the paper in Section 6.

2 Preliminaries

We begin with a review of material from the literature. We summarize

- Agda and language features relevant to our work: rewrite rules and opaque blocks;
- multi-sorted and scoped syntax in Agda and
- the σ-calculus modeled via the de Bruijn algebra.

All Agda code presented in this paper can be found in the supplement.

2.1 Agda and Relevant Language Features

Agda is a dependently typed functional language and proof assistant based on Martin-Löf type theory [20], where proofs are verified by type checking. Proofs are significantly easier when terms are *definitionally equal* (i.e., reduce to the same term). When terms are not definitionally equal, users must resort to manual equational reasoning using propositional equality, which is a cumbersome process as Agda often lacks automated proof-discharge tactics. Maximizing definitional equalities thus directly reduces manual effort.

Beyond standard features such as indexed inductive types and dependent pattern matching [11], this work is based on two more advanced language features.

Rewrite Rules

User-defined rewrite rules [12, 10] extend Agda's normalization-by-evaluation approach for definitional equalities by treating propositional equalities as reduction rules.

A function is eligible for rewriting if it is of the following form:

$$\mathsf{eq} : (x_1 : A_1) \ldots \to \mathsf{f}\, p_1 \ldots p_n \equiv v$$

In addition it needs to satisfy the following conditions:

1. The head f is a postulate, defined function, or constructor with distinct variable type parameters.

2. Every variable x_i appears in a pattern position among $p_1, \ldots, p_n$[1].
3. The left-hand side is stuck.

For example, the right identity of addition satisfies these criteria:

```
+-idr : ∀ n → n + 0 ≡ n
+-idr zero    = refl
+-idr (suc n) = cong suc (+-idr n)
```

Once the rule is registered via {-# REWRITE +-idr #-}, Agda treats $n + 0$ as definitionally equal to n:

```
_ : ∀ {n} → n + 0 ≡ n
_ = refl
```

In addition to the syntactic restrictions, users must ensure that the rewrite rule set as a whole is terminating and confluent to retain logical consistency.

Termination is not verified, but also not a critical property [12]. Agda provides confluence checks via `--confluence-check` and `--local-confluence-check`. Given that the rewritten rules are supported by proofs, and we asume that the rewrite system is terminating, local confluence is enough to preserve consistency [12].

Opaque Blocks

Opaque definitions [15] let the user treat code as non-unfolding symbols. This feature is typically used to improve type checking times, because normally we want all definitions to unfold, to produce as many definitional equalities as possible. Their use case in this work will be to declare relevant definitions opaque, such that the stuck symbols qualify for rewriting.

Definitions within an opaque block are visible but treated abstractly during type checking:

```
opaque
  forty-two : Nat
  forty-two = 42
```

Outside this block, forty-two does not reduce to 42:

```
_ : forty-two ≡ 42
_ = {!   !} -- not provable.
```

However, the unfolding keyword can be used to expose the computational behavior of a symbol when needed:

```
opaque
  unfolding forty-two

  _ : forty-two ≡ 42
  _ = refl
```

```
data Kind : Set where
  ⋆ : Kind

data Type (n : Nat) : Set where
  ‘_          : Fin n → Type n
  ∀[α:_]_     : Type (suc n) → Type n
  _⇒_         : Type n → Type n → Type n

data Expr (n m : Nat) : Set where
  ‘_     : Fin m → Expr n m
  λx_    : Expr n (suc m) → Expr n m
  Λα_    : Expr (suc n) m → Expr n m
  _·_    : Expr n m → Expr n m → Expr n m
  _•_    : Expr n m → Type n → Expr n m
```

```
data Sort : Set where
  expr type kind : Sort

Scope = List Sort

data Mode : Set where V T : Mode

data _⊢[_]_ : Scope → Mode → Sort → Set

_⊢_ = _⊢[ T ]_
_∋_ = _⊢[ V ]_

data _⊢[_]_ where
  zero      : (s :: S) ∋ s
  suc       : S ∋ s → (s′ :: S) ∋ s
  ‘_        : S ∋ s → S ⊢ s
  λx_       : (expr :: S) ⊢ expr → S ⊢ expr
  Λα_       : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_]_   : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  _·_       : S ⊢ expr → S ⊢ expr → S ⊢ expr
  _•_       : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_       : S ⊢ type → S ⊢ type → S ⊢ type
  *         : S ⊢ kind
```

Figure 1 Classically Scoped vs. Multi-Sorted Scoped Syntax

2.2 Multi-Sorted and Scoped Syntax

Many programming languages feature multiple syntactic categories. For instance System F [14, 22] has three syntactic sorts: expressions, types, and kinds [2]. We use intrinsically scoped syntax, to prevent the construction of ill-scoped terms from our syntax. Figure 1 (left) shows a standard implementation: types are indexed by the amount of type variables, and expressions by both the amount of type and expression variables. Variables use de Bruijn indices of type Fin n, that is, natural numbers strictly less than the number of free variables.

We see scoped syntax as the optimal balance between fully extrinsic syntax and intrinsically typed syntax, in which case the expression syntax is indexed directly over contexts and types. It is powerful enough to detect scoping related errors, but one does not need to commit to full blown intrinsic typing, which can be difficult to work with for more complex systems.

Alas, the standard representation requires us to define many substitution operations across the different sorts (e.g., type-in-type, type-in-expr, expr-in-expr). This representation is chosen by Autosubst 2, and explains why they need vector substitutions, that bundle substitutions over the different sorts alongside additional normalization rules.

To avoid the combinatorial explosion of substitutions over different syntactic sorts, we unify the syntax in two orthogonal dimensions, presented also in Figure 1 (right). Variables

[1] The Agda documentation includes a definition for pattern positions: `https://Agda.readthedocs.io/en/latest/language/rewriting.html`

[2] Technically kinding for System F is trivial and could be omitted.

Traversal Rules

$$\begin{array}{lll} \mathsf{lam} & (\lambda.\, t) \cdots \sigma & = \lambda.(t \cdots (\uparrow\sigma)) \\ \mathsf{app} & (t_1\ t_2) \cdots \sigma & = (t_1 \cdots \sigma)\ (t_2 \cdots \sigma) \end{array}$$

Monad Rules

$$\begin{array}{lll} \mathsf{right\text{-}id} & t \cdots \mathsf{id} & = t \\ \mathsf{comp} & (t \cdots \sigma_1) \cdots \sigma_2 & = t \cdots (\sigma_1 ; \sigma_2) \end{array}$$

Interaction Rules

$$\begin{array}{rll} \mathsf{id_L} & \mathsf{id}\ ;\ \sigma & = \sigma \\ \mathsf{id_R} & \sigma\ ;\ \mathsf{id} & = \sigma \\ \mathsf{assoc} & (\sigma_1\ ;\ \sigma_2)\ ;\ \sigma_3 & = \sigma_1\ ;\ (\sigma_2\ ;\ \sigma_3) \\ \mathsf{dist} & (t \cdot \sigma_1)\ ;\ \sigma_2 & = (t \cdots \sigma_2) \cdot (\sigma_1\ ;\ \sigma_2) \\ \mathsf{interaction} & \mathsf{wk}\ ;\ (t \cdot \sigma) & = \sigma \\ \eta\text{-}\mathsf{id} & (\mathsf{zero}) \cdot \mathsf{wk} & = \mathsf{id} \\ \eta\text{-}\mathsf{law} & (\mathsf{zero} \cdots \sigma) \cdot (\mathsf{wk}\ ;\ \sigma) & = \sigma \end{array}$$

Figure 2 The Sigma Calculus Rules

and terms form a single datatype $S \vdash [\ m\ ]\ s$ indexed by a Mode m, a target Sort s and a list of sorts Scope S. Modes distinguish constructors for variables and terms. Variables are represented across multiple sorts simultaneously by natural numbers of type $S \ni s$ and act as de Bruijn indices. The length of S encodes the amount of variables in scope and the position of s in S directly corresponds to the de Bruijn index of the variable. Using the syntax unified over sorts, we can later define renamings and substitutions that operate uniformly across all sorts, thereby avoiding vector substitutions. Unifying over modes, allows us to state some σ-calculus laws parametric in the mode.

2.3 The Sigma Calculus modeled via the de Bruijn Algebra

We review the variant of the σ-calculus modeled via the de Bruijn algebra which also appears in the first version of Autosubst [26], without first-class renamings. Variables x represent natural numbers and terms $t := x \mid \lambda.\, t \mid t_1\, t_2$ include the standard lambda calculus constructors as an example for any single sorted, unscoped syntax with binding. A substitution σ maps variables to terms. It helps to view a substitution as a list $[t_0, t_1, \ldots]$, where the index of the term in the list corresponds to the de Bruijn variable it replaces. Binders make things complicated and require support for a shift operation to increment de Bruijn indices, when substitutions containing variables are pushed under binders. This perspective motivates the semantic definitions for the extension operation $t \cdot \sigma$, the identity id and the weakening wk primitives, treating substitutions as functions:

$$\begin{array}{ll} (t \cdot \sigma)(\mathsf{zero}) := t & \mathsf{id}(x) := x \\ (t \cdot \sigma)(\mathsf{suc}\, x) := \sigma(x) & \mathsf{wk}(x) := \mathsf{suc}\, x \end{array}$$

We then define the application of a substitution to a term $\cdots$, which we refer to as instantiation, mutually recursively with composition and the lifting operator which pushes substitutions under binders using shifting:

$$\begin{array}{ll} (\lambda.\, t) \cdots \sigma = \lambda.(t \cdots (\uparrow\sigma)) & (\sigma_1 \fatsemi \sigma_2)(x) = \sigma_1(x) \cdots \sigma_2 \\ (t_1\ t_2) \cdots \sigma = (t_1 \cdots \sigma)\ (t_2 \cdots \sigma) & \uparrow\sigma := \mathsf{zero} \cdot (\sigma \fatsemi \mathsf{wk}) \end{array}$$

Lifting preserves the bound variable while it weakens the terms of the substitution to maintain correct scoping. Composition retrieves the term from the first substitution and then instantiates the resulting term with the second. Instantiation traverses the term homomorphically to push the substitution to the leaves.

Figure 2 presents the accompanying rewrite system. It traditionally also includes the clauses of the instantiation function. We follow Stark [28] and categorize the rules into three groups.

- **Traversal rules** govern propagation through terms and under binders.
- **Monad rules** define the behavior of substitutions applied to terms.
- **Interaction rules** describe equalities between substitution expressions.

We refer the reader to the original σ_{SP}-calculus paper [25] for a detailed description of the rule set selection, and to Schäfer et al [25] for the reason these rules suffice to normalize all substitution expressions.

3 Rewriting the Sigma Calculus

The definition of instantiation by mutual recursion with composition complicates termination. Agda cannot determine that this definition is structurally recursive (and also lacks mechanisms to justify termination by well-founded recursion), so we follow Adams [3] to resolve the issue by first defining instantiation with renamings. We then extend the equational theory to handle first-class renamings, and also to connect renamings with substitutions, with the rules from Autosubst 2. We use the multi-sorted and scoped version of System F from Figure 1 as an example for any scoped syntax. The approach described in this section directly translates to any multi-sorted and scoped syntax as well as simple intrinsically typed syntax where the type index of the syntax is of type Set.

3.1 Renaming and Substitution

The definitions for renaming and substitution are presented in Figure 3. The definitions are mostly standard, but we will block the reduction for most of the renaming and substitution primitives, and describe their behavior later through rewrite rules.

We begin with the type alias for renamings $_\rightarrow^R_$: renamings are functions that map variables of sort s from scope S_1 to variables of the same sort in scope S_2. We then define the renaming primitives identity id, weakening wk and forward composition $_\circ_$. The identity renaming is just the identity function, that ignores the explicit sort argument, which we could not leave implicit, because Agda often fails to infer it, complicating proofs unnecessarily. Weakening applies the suc constructor to all variables of a renaming, and composition of two renamings is defined as standard forward function composition. Extension $_\cdot^R_$ inserts the respective variable for de Bruijn index zero into the renaming. Lifting $_\uparrow^R_$ is defined in terms of extension with variable zero and composition with the weakening renaming. We do not mark lifting opaque, because it is merely a alias for a term constructed from other primitives. The traversal operation $_\cdots^R_$ propagates the renaming through the nodes to the leaves and applies lifting when it passes under binders. We define instantiation across both modes for reasons explained later.

Substitutions are functions from variables to terms instead of variables. The operator $\langle_\rangle$ embeds any renaming into a substitution by wrapping all variables with the variable constructor. With the embedding operator we define aliases for the identity substitution id^S and weakening substitution wk^S directly from renamings. To prevent Agda from rejecting rewrite rules due to reductions on the left-hand side of equations, we INLINE these symbols.

Extension $_\cdot^S_$ is defined by case analysis on the incoming variable and maps the term we extend with to variable zero. We use instantiation with the weakening renaming to define the lifting of substitutions $_\uparrow^S_$, thereby satisfying the requirements of structural recursion.

```
_→ᴿ_ : Scope → Scope → Set
S₁ →ᴿ S₂ = ∀ s → S₁ ∋ s → S₂ ∋ s
opaque
  idᴿ : S →ᴿ S
  idᴿ _ x = x
  wkᴿ : ∀ s → S →ᴿ (s ∷ S)
  wkᴿ _ _ = suc
  _∘_ : S₁ →ᴿ S₂ → S₂ →ᴿ S₃ →
    S₁ →ᴿ S₃
  (ρ₁ ∘ ρ₂) _ x = ρ₂ _ (ρ₁ _ x)
  _·ᴿ_ : S₂ ∋ s → S₁ →ᴿ S₂ →
    (s ∷ S₁) →ᴿ S₂
  (x ·ᴿ ρ) _ zero = x
  (_ ·ᴿ ρ) _ (suc x) = ρ _ x
_↑ᴿ_ : (S₁ →ᴿ S₂) → ∀ s →
  ((s ∷ S₁) →ᴿ (s ∷ S₂))
(ρ ↑ᴿ _) = zero ·ᴿ (ρ ∘ (wkᴿ _))
opaque
  _⋯ᴿ_ : S₁ ⊢[ m ] s → S₁ →ᴿ S₂ →
    S₂ ⊢[ m ] s
  _⋯ᴿ_ {m = V} x ρ = ρ _ x
  (` x)        ⋯ᴿ ρ = ` ρ _ x
  (λx e)       ⋯ᴿ ρ = λx (e ⋯ᴿ (ρ ↑ᴿ _))
  (Λα e)       ⋯ᴿ ρ = Λα (e ⋯ᴿ (ρ ↑ᴿ _))
  (∀[α: k ] t) ⋯ᴿ ρ = ∀[α: k ⋯ᴿ ρ ]
                      (t ⋯ᴿ (ρ ↑ᴿ _))
  (e₁ · e₂)    ⋯ᴿ ρ = (e₁ ⋯ᴿ ρ) · (e₂ ⋯ᴿ ρ)
  (e • t)      ⋯ᴿ ρ = (e ⋯ᴿ ρ) • (t ⋯ᴿ ρ)
  (t₁ ⇒ t₂)    ⋯ᴿ ρ = (t₁ ⋯ᴿ ρ) ⇒ (t₂ ⋯ᴿ ρ)
  *            ⋯ᴿ ρ = *
```

```
_→ˢ_ : Scope → Scope → Set
S₁ →ˢ S₂ = ∀ s → S₁ ∋ s → S₂ ⊢ s
opaque
  ⟨_⟩ : S₁ →ᴿ S₂ → S₁ →ˢ S₂
  ⟨ ρ ⟩ _ x = ` ρ _ x
idˢ : S →ˢ S
idˢ = ⟨ idᴿ ⟩
{-# INLINE idˢ #-}
wkˢ : ∀ s → S →ˢ (s ∷ S)
wkˢ _ = ⟨ wkᴿ _ ⟩
{-# INLINE wkˢ #-}
opaque
  unfolding _⋯ᴿ_
  _·ˢ_ : S₂ ⊢ s → S₁ →ˢ S₂ → (s ∷ S₁) →ˢ S₂
  (t ·ˢ σ) _ zero = t
  (t ·ˢ σ) _ (suc x) = σ _ x
  _↑ˢ_ : S₁ →ˢ S₂ → ∀ s → (s ∷ S₁) →ˢ (s ∷ S₂)
  σ ↑ˢ s = (` zero) ·ˢ λ _ x → (σ _ x) ⋯ᴿ wkᴿ _
  _⋯ˢ_ : S₁ ⊢[ m ] s → S₁ →ˢ S₂ → S₂ ⊢ s
  _⋯ˢ_ {m = V} x σ = σ _ x
  (` x)        ⋯ˢ σ = σ _ x
  (λx e)       ⋯ˢ σ = λx (e ⋯ˢ (σ ↑ˢ _))
  (Λα e)       ⋯ˢ σ = Λα (e ⋯ˢ (σ ↑ˢ _))
  (∀[α: k ] t) ⋯ˢ σ = ∀[α: k ⋯ˢ σ ] (t ⋯ˢ (σ ↑ˢ _))
  (e₁ · e₂)    ⋯ˢ σ = (e₁ ⋯ˢ σ) · (e₂ ⋯ˢ σ)
  (e • t)      ⋯ˢ σ = (e ⋯ˢ σ) • (t ⋯ˢ σ)
  (t₁ ⇒ t₂)    ⋯ˢ σ = (t₁ ⋯ˢ σ) ⇒ (t₂ ⋯ˢ σ)
  *            ⋯ˢ σ = *
_⨾_ : S₁ →ˢ S₂ → S₂ →ˢ S₃ → S₁ →ˢ S₃
(σ₁ ⨾ σ₂) _ x = (σ₁ _ x) ⋯ˢ σ₂
```

Figure 3 The Primitives for Renaming and Substitution

In contrast to the lifting operator for renamings, we keep its computational behavior hidden, and later rewrite it in terms of substitution composition. Instantiation with a substitution _⋯ˢ_ works analogous to the case of renamings and we also design it to work for variables and terms simultaneously. Finally we define the forward composition of substitutions _⨾_ via instantiation.

The definitions of instantiation over both modes, allows us to perform lookup of a variables in substitutions (or renamings) also by using of the stuck symbol for instantiation (x ⋯ˢ σ), instead of function application (σ _ x). Because we need to state the σ-calculus rules only in terms of stuck symbols, without using lambdas or function application, we require an additional symbol for variable lookup. We go into more detail in Section 3.4.

```
-- definitional rules
def-·^S-zero : zero ⋯^S (t ·^S σ)  ≡ t
def-·^S-suc  : suc x ⋯^S (t ·^S σ) ≡ x ⋯^S σ
def-⨾ : (x ⋯^S (σ₁ ⨾ σ₂)) ≡ ((x ⋯^S σ₁) ⋯^S σ₂)
def-↑^S : σ ↑^S s ≡ (‘ zero) ·^S (σ ⨾ wk^S _)
-- interaction rules
assoc : (σ₁ ⨾ σ₂) ⨾ σ₃ ≡ σ₁ ⨾ (σ₂ ⨾ σ₃)
dist : (t ·^S σ₁) ⨾ σ₂ ≡ ((t ⋯^S σ₂) ·^S (σ₁ ⨾ σ₂))
interact          : wk^S s ⨾ (t ·^S σ) ≡ σ
comp-id_r         : σ ⨾ id^S ≡ σ
comp-id_l         : id^S ⨾ σ ≡ σ
η-id : (‘ zero {s} {S}) ·^S (wk^S _)   ≡ id^S
η-law : (zero ⋯^S σ) ·^S (wk^S _ ⨾ σ) ≡ σ
-- complete rewrite system
{-# REWRITE
def-·^S-zero def-·^S-suc def-↑^S def-⨾
assoc dist interact
comp-id_r comp-id_l η-id η-law
inst-x inst-λ inst-Λ inst-∀ inst-· inst-•
inst-⇒ inst-*
right-id
compositionality^RR compositionality^RS
compositionality^SR compositionality^SS
coincidence coincidence-fold

def-id def-wk def-·^R-zero def-·^R-suc def-∘
assoc^R dist^R interact^R
comp-id_r^R comp-id_l^R η-id^R η-law^R
inst^R-x inst^R-λ inst^R-Λ inst^R-∀ inst^R-· inst^R-•
inst^R-⇒ inst^R-*
coincidence-var
#-}
```

```
-- coincidence rules
coincidence : ∀ (t : S ⊢ s) →
  t ⋯^S ⟨ ρ ⟩ ≡ (t ⋯^R ρ)
coincidence-fold :
  ⟨ ρ ↑^R s ⟩ ⨾ ((t ⋯^R ρ) ·^S id^S) ≡ (t ⋯^R ρ) ·^S ⟨ ρ ⟩
-- traversal rules
inst-x  : (‘ x)        ⋯^S σ ≡ x ⋯^S σ
inst-λ  : (λx e)       ⋯^S σ ≡
  λx (e ⋯^S (σ ↑^S _))
inst-Λ  : (Λα e)       ⋯^S σ ≡
  Λα (e ⋯^S (σ ↑^S _))
inst-∀  : (∀[α: k ] t) ⋯^S σ ≡
  ∀[α: k ⋯^S σ ] (t ⋯^S (σ ↑^S _))
inst-·  : (e₁ · e₂)    ⋯^S σ ≡
  (e₁ ⋯^S σ) · (e₂ ⋯^S σ)
inst-•  : (e • t)      ⋯^S σ ≡
  (e ⋯^S σ) • (t ⋯^S σ)
inst-⇒ : (t₁ ⇒ t₂)     ⋯^S σ ≡
  (t₁ ⋯^S σ) ⇒ (t₂ ⋯^S σ)
inst-*  : *            ⋯^S σ ≡ *
-- monad rules
right-id : ∀ (t : S ⊢ s) → t ⋯^R id^R ≡ t
compositionality^RR : ∀ (t : S ⊢ s) →
  (t ⋯^R ρ₁) ⋯^R ρ₂ ≡ t ⋯^R (ρ₁ ∘ ρ₂)
compositionality^RS : ∀ (t : S ⊢ s) →
  (t ⋯^R ρ₁) ⋯^S σ₂ ≡ t ⋯^S (⟨ ρ₁ ⟩ ⨾ σ₂)
compositionality^SR : ∀ (t : S ⊢ s) →
  (t ⋯^S σ₁) ⋯^R ρ₂ ≡ t ⋯^S (σ₁ ⨾ ⟨ ρ₂ ⟩)
compositionality^SS : ∀ (t : S ⊢ s) →
  (t ⋯^S σ₁) ⋯^S σ₂ ≡ t ⋯^S (σ₁ ⨾ σ₂)
```

Figure 4 The Rewrite System For the Sigma Calculus with First-Class Renamings

3.2 The Sigma Calculus Rules

Figure 4 presents the part of σ-calculus rules with first-class renamings as Agda equalities admissible for rewriting. All rules from the second paragraph of the REWRITE block were omitted from the figure and can be found in the supplement.

Agda accepts the rule set as locally confluent which is important for safety. We are also able to prove all the rules before rewriting them, to guarantee consistency. We now discuss the changes to the standard σ-calculus variant from Section 2.3.

The **Definitional rules** include definitions we blocked earlier in modified form. We include the clauses from the definition of extension but use the instantiation operator to handle lookup of variables in substitutions instead of function application. Similarly, lookup in composed substitutions is defined in terms of the instantiation operator. We rewrite the lifting operator in terms of composition with the weakening substitution, which we were not possible to do earlier, due to the aforementioned termination issues. This change allows us to

draw on the complete theory of the σ-calculus, when solely reasoning about substitutions. To save space, we have omitted the definitional rules for renamings, but they follow analogously.

The **Interaction rules** are exactly those presented in Section 2.3. In the figure we have omitted a copy of the interaction rules for renamings, which we also include in the rewrite system.

The **Coincidence rules** emerge from the interaction between renamings and substitutions [29]. They allow us to recover some renamings from their embedding as substitutions. The extraction of renamings from deeply nested substitutions in general requires a dedicated solving strategy [29]. In addition to the obvious coincidence rule, we require a specific coincidence variant coincidence-fold for the proof of subject reduction for System F later. However, others proofs might require more coincidence lemmas. For instance, proving subject reduction in general requires the inclusion of lemmas in the style of coincidence-fold with n extensions where n is the maximum arity of binders in the object language. The inclusion of more coincidence lemmas usually does not affect confluence.

The **Traversal rules** mirror the clauses from the definitions of instantiation in the previous section directly. We include all clauses except the one for variables. This exclusion is important because we wanted the term $x \cdots^S \sigma$ to remain stuck, so we can apply rewrite rules where instantiation of variables is the head symbol. We have omitted the analogous traversal rules for renamings.

We extend the **Monad rules** to handle all four combinations of instantiation with renamings and substitutions. We state the right identity law only for renamings because the instantiation with the identity substitution reduces to instantiation with the identity renaming via coincidence.

The proofs of the rules are standard, and we refer the interested reader to the supplement. Within the proofs we unfold the opaque definitions and make use of function extensionality which is conservative over the theory underlying Agda [16].

3.3 An Example Proof with the Rewritten Sigma Calculus

The rules of the σ-calculus apply automatically throughout the subsequent code. We briefly demonstrate this with a proof of subject reduction where we never need to explicitly manage substitutions. The proof is carried out for a standard logical type system $_\vdash_:_$ and call-by-value small-step operational semantics $_\hookrightarrow_$. We refer to the supplement for the definitions of both.

To establish subject reduction, we first prove that renaming and substitution preserve typing. The left side of Figure 5 displays the proofs for the substitution-preserves-typing lemma $_\vdash\cdots^S_$. We omit the analogous proof for renamings.

The signature of the proof relies on the definition for well typed substitutions:

$$_:_\rightarrow^S_ : S_1 \rightarrow^S S_2 \rightarrow (\Gamma_1 : \mathsf{Ctx}\ S_1) \rightarrow (\Gamma_2 : \mathsf{Ctx}\ S_2) \rightarrow \mathsf{Set}$$

Contexts $\mathsf{Ctx}\ S$ are maps from scoped variables to scoped types. Well typed substitutions are then defined as functions from variables x typed in Γ_1 to terms $x \cdots^S \sigma$ typed in Γ_2.

The preservation result for renamings is used to prove that the lifting of a substitution preserves typing $\vdash\uparrow^S$ and subsequently used inside the substitution-preserves-typing proof, again respecting the termination constraints. The variable case requires a lookup of the well typed variable inside the well typed substitution. We apply the preservation lemma for lifting substitutions when we pass under a binder. All cases follow without any manipulation of substitutions. In particular we do not need to manually move the substitution under a

```
_⊢⋯ˢ[_]_ :
  Γ₁ ⊢ t : t′ →
  (σ : S₁ →ˢ S₂) →
  σ : Γ₁ →ˢ Γ₂ →
  Γ₂ ⊢ (t ⋯ˢ σ) : (t′ ⋯ˢ σ)
(⊢‘ ⊢x)          ⊢⋯ˢ[ σ ] ⊢σ =
  ⊢σ _ _ _ ⊢x
(⊢λ ⊢e)          ⊢⋯ˢ[ σ ] ⊢σ =
  ⊢λ (⊢e ⊢⋯ˢ[ σ ↑ˢ _ ] (⊢↑ˢ[ σ ] ⊢σ) _)
(⊢Λ ⊢e)          ⊢⋯ˢ[ σ ] ⊢σ =
  ⊢Λ (⊢e ⊢⋯ˢ[ σ ↑ˢ _ ] (⊢↑ˢ[ σ ] ⊢σ) _)
(⊢· ⊢e₁ ⊢e₂)     ⊢⋯ˢ[ σ ] ⊢σ =
  ⊢· (⊢e₁ ⊢⋯ˢ[ σ ] ⊢σ) (⊢e₂ ⊢⋯ˢ[ σ ] ⊢σ)
(⊢• ⊢e ⊢t ⊢t’)   ⊢⋯ˢ[ σ ] ⊢σ =
  ⊢• (⊢e ⊢⋯ˢ[ σ ] ⊢σ) (⊢t ⊢⋯ˢ[ σ ] ⊢σ)
  (⊢t’ ⊢⋯ˢ[ σ ↑ˢ _ ] (⊢↑ˢ[ σ ] ⊢σ) _)
⊢*               ⊢⋯ˢ[ σ ] ⊢σ = ⊢*
```

```
sr :
  Γ ⊢ e : t →
  e ↪ e′ →
  Γ ⊢ e′ : t
sr (⊢· {e₂ = e₂} (⊢λ ⊢e₁) ⊢e₂) (β-λ v₂) =
  ⊢e₁ ⊢⋯ˢ[ e₂ ·ˢ idˢ ] (⊢[] ⊢e₂)
sr (⊢• {t = t} (⊢Λ ⊢e) ⊢t ⊢t’) β-Λ =
  ⊢e ⊢⋯ˢ[ t ·ˢ idˢ ] (⊢[] ⊢t)
sr (⊢· ⊢e₁ ⊢e₂) (ξ-·₁ e₁↪e) =
  ⊢· (sr ⊢e₁ e₁↪e) ⊢e₂
sr (⊢· ⊢e₁ ⊢e₂) (ξ-·₂ e₂↪e x) =
  ⊢· ⊢e₁ (sr ⊢e₂ e₂↪e)
sr (⊢• ⊢e ⊢t ⊢t’) (ξ-• e↪e’) =
  ⊢• (sr ⊢e e↪e’) ⊢t ⊢t’
```

Figure 5 Subject Reduction for System F

weakening inside the cases λx_ and Λα_ or under a singleton substitution in the case of type application _•_ which are normally required in standard proofs.

Subject reduction sr directly follows from the substitution preserves typing lemma, shown on the right of Figure 5. Inside the β-reduction cases ⊢[] denotes the well typed single substitution which can be derived directly from the substitution preserves typing lemma. The rewrite rules automatically handle the otherwise required application of interaction rules in the cases of β-reductions.

3.4 An Examination of the Issue with Unfolding Definitions

Blocking the reduction behavior of renaming and substitution primitives allowed us to successfully rewrite all σ-calculus rules. We now explain why this approach works, and more specifically why other attempts did not work.

The primary issue is that composition unfolds to lambda abstractions. If we unfold the definition for the composition operator, then for instance the left hand side of comp-id$_l$ rule would reduce to $(\lambda\ _\ x \to \sigma\ _\ x \cdots^S (\lambda\ _ \to \text{‘}_))$, violating the constraint that rewrite rules must have stuck symbols on the left-hand side. However, when we block the reduction of composition, we still need to be able to describe how variables are looked up inside two composed substitutions.

We tried to keep using function application to describe variable lookup, that is, rewriting the following: $(\sigma_1 \fatsemi \sigma_2)\ _\ x \equiv (x \cdots^S \sigma_1) \cdots^S \sigma_2$. This equation effectively re-establishes the computational behavior of composition and to our surprise, Agda accepts this rule together with the interaction rules, such as comp-id$_l$, but it fails to perform higher-order pattern unification correctly afterwards. More specifically, it does not apply the η-laws anymore, even though manual application using propositional equality reasoning works. We argue Agda should reject such a construction.

Instead we use the instantiation operator to perform variable lookup. Doing so requires us to propagate this change through all rules, using instantiation every time we wanted to use function application for variable lookup.

```
kind : Type
type : Type
expr : Type

-- the constructors for kind
star : kind

-- the constructors for type
arr : type -> type -> type
all : kind -> (type -> type) -> type

-- the constructors for expr
app  : expr -> expr -> expr
lam  : type -> (expr -> expr) -> expr
tapp : expr -> type -> expr
tlam : kind -> (type -> expr) -> expr
```

Figure 6 Higher-order abstract syntax specification for System F

We note that, while we are still able to use function application or build renamings and substitutions using lambdas, we then obtain fewer definitional equalities. We thus recommend only to use the instantiation operator for variable lookup and only the two already defined composition operators.

Using Vectors instead of Functions

We also considered an alternative representation for substitutions based on a vector-like data type. This data type would at least need to include an extension constructor, to fill the vector with terms. Because composition is then defined via case distinction on the vector, it correctly behaves as a stuck symbol suitable for rewriting. However, with extension as constructor the η-rules are not instantiable with proofs, because this would require that the extension constructor has computational behavior.

Removing the η-rules from the equational theory preserves confluence, but removes important definitional equalities. However, the vector representation could maybe still be useful if one attempts to formalize type-theory-in-type-theory [7], where a representation of substitutions as functions is not possible.

3.5 Discussion

The rules presented in Section 3.2 are an extension of the classic σ-calculus rules to a setting with renamings, closely following the work done by the Autosubst 2 authors. After careful construction, marking the relevant symbols opaque, they fulfill the requirements to be registered as rewrite rules. Moreover Agda can see that the rule set preserves confluence.

The original σ-calculus treats the symbols purely syntactic, also describing the behavior of lookup with use of the instantiation operator. Then the de Bruijn algebra provided a semantic model for the σ-calculus using functions, enabling proofs for the σ-calculus rules and applicability for automated propositional equality reasoning. In this work we do both: Using the semantic model to prove the laws but also treating the symbols syntactically to predicably apply rewrite rules.

4 Ongoing and Future Work

4.1 Code Generation

In addition to the theoretical work, Autosubst 2 also features external code generation from higher-order abstract syntax (HOAS) descriptions. More specifically, they generate the scoped syntax data types and all substitution lemmas that require induction over the syntax for the Rocq proof assistant.

To showcase our applicability to custom syntax descriptions, we also provide a code generation script that translates HOAS descriptions to Agda code, which includes the σ-calculus rules ready for rewriting. The corresponding HOAS description for our SystemF example can be found in Figure 6. We are already able to support some of the features that Autosubst 2 supports:

- Vector substitutions are unnecessary due to multi-sorted syntax.
- Mutual recursive syntax definitions work out of the box with multi-sorted syntax.
- Binders that bind multiple variables are supported by a generalized lifting operation that allows to lift by lists of sorts, instead of only a single sort.
- Inline Agda code (e.g. lists or products of terms) can already be encoded using additional sorts, but one loses the support for lemmas from the standard library.
- Distinction between sorts with binders and those without is not yet supported, but could be encoded with an additional binary index to the Sort data type, that separates sorts of the two categories.
- Variadic syntax is also not yet supported, but could be encoded in the future using arguments to the sort constructors [23]

Our code generation script can be found in the supplement, but represents merely a proof of concept for a standalone library. The script includes a parser for HOAS descriptions and ad-hoc code generation. It is written in Python with strict type-checking enabled and includes all the boilerplate Agda code, so it can serve as a single-file standalone tool for others to try. We include a dozen Agda files which we generated from HOAS descriptions found in the examples directory of the Autosubst 2 git repository.

4.2 Opaque Definitions for Rewriting

The construction from the previous section reveals a broader concept that could serve as a general recipe for integrating normalization for suitable algebraic structures into proof assistants:

- Define symbols for the structure.
- Establish a corresponding confluent rewrite system.
- Instantiate the structure with an underlying model to prove the laws of the rewrite system inside opaque blocks.
- With the reduction of the model hidden, REWRITE the algebraic equations.

To illustrate this, we consider the example of classical sets. When reasoning over sets in Agda, users often resort to work with lists up to permutation and a uniqueness predicate, which is tedious to work with and requires lots of uninteresting code. This overhead is mostly not an issue in tactic-based provers, that feature dedicated solvers for this work.

The characteristic function for sets makes it possible to translate statements about sets to statements about functions. We shortly outline how our approach could be adopted to

provide built-in normalization for sets using their characteristic function as the underlying model.

Following the idea above, the type of sets would be defined as: $\mathcal{S} = \mathcal{A} \to \mathsf{Bool}$ for some $\mathcal{A} : \mathsf{Set}\ \ell$. Using this model we can define set operations as opaque functions:

```
opaque
  ∅ : 𝒮
  ∅ a = false

  _∪_ : 𝒮 → 𝒮 → 𝒮
  (A ∪ B) a = A a ∨ B a

  -- ...
```

Although this model is probably even worse to work with than lists up to permutation, we can define all set operations and also instantiate a suitable rewrite system with proofs. This rewrite system might include for example a right identity law of the form: $(A \cup \emptyset) \equiv A$. Due to the opaquely defined operations, this equation qualifies as rewrite rule.

A good starting point for a suitable rewrite system is given by the boolean ring interpretation of sets [17]. This system however, requires rewriting modulo AC (associativity and commutativity). Other provers, such as Dedukti [8], which is purely based on rewrite rules, already support AC rewriting and we argue that Agda would also greatly benefit from an extension with rewriting modulo AC. Safe built-in normalization for common algebraic structures would improve the user experience of Agda and might allow it to close the gap to tactic-based provers regarding automation of such proofs.

4.3 Implications for Intriniscally Typed Syntax

Complex intrinsically typed syntax, where substitution already appears as an operation on the type index of the syntax, complicates proofs significantly. Because substitutions without rewriting enjoy few definitional equalities, we need to apply general properties of the transfer lemma (subst in Agda) every time we manipulate proofs over expressions that involve type substitution. This issue is commonly referred to as transfer hell (or transport hell).

The fact that complex intrinsically typed syntax ends up in transfer hell is long-standing issue [9]. An elaborate example close to our work in Agda is given by the construction of a binary logical relation for intrinsically typed System F [24], where the authors spend thousands of lines reasoning about uninteresting manipulation of transfers. More specifically, while reasoning over type-in-type and expr-in-expr substitutions is still easy in this scenario, proving lemmas over type-in-expr substitution is complicated tremendously, because type substitution also affects the index of the expression that is substituted in. After rewriting the σ-calculus rules for type substitutions, their problems in the proofs for expression substitutions should effectively be gone completely.

Rewriting the σ-calculus rules on the syntax for the typing index should easily generalize to other intrinsically typed formalizations and render explicit transfers unnecessary.

5 Related Work

We already discussed Autosubst and related research [29, 26, 28], as well as user-defined rewrite rules [12, 10, 18] and opaque definitions [15], which are both present in Agda and Rocq, extensively throughout this work.

In their recent work on local rewrite rules [19], the authors also rewrite σ-calculus rules with support for first-class renamings in Rocq, similar to our approach. They also provide a model for renamings and substitutions as functions, but they are unable to rewrite all laws given the underlying model. This is because in their case the instantiation for the set of equations must hold definitionally and cannot be propositional to maintain consistency in the presence of local rewrite rules. Rocq also allows for global rewrite rules [18], and we expect our approach can be translated to Rocq. However, Rocq lacks the ability to instantiate the rewrite rules with proofs, and we would then lose consistency guarantees.

Another line of research concerns generic programming approaches [4, 5], that allow the construction of syntax from general building blocks that support sums, products and binder introduction. This allows for a generic structure that abstracts over renamings and substitutions. The generic syntax approach translates to a scoped and multi-sorted setting [23]. Importantly, we believe generic programming and rewriting the σ-calculus rules are orthogonal and our rewrite system could be applied to the universe of syntaxes with binding, thereby avoiding external code generation.

The Abstract Binding Trees library for Agda shares a similar goal with Autosubst and is used for instance in Siek's work [27]. It permits the construction of an object language from a generic syntax similar to the universe of syntaxes with binding, but it only supports extrinsic scoping and single sorted syntaxes. The library mainly focuses on generic theorems about substitution and abstractions for arbitrary predicates over the object language. Interestingly, the library also offers incomplete experimental support for rewriting using σ-calculus laws but misses rules of the σ-calculus and thus does not pass the confluence check.

Wadler experiments with explicit substitutions and implements weakening as a constructor in the syntax [30], thereby successfully avoiding renamings. He rewrites some of the sigma calculus laws but cannot pass the confluence checker. Because weakening is a constructor, variables in his representation can be represented by multiple terms that are not definitionally equal which results in the loss of definitional equalities. Furthermore, he represents substitutions using vectors, and thus would not be able to support the full σ-calculus rule set with proofs.

Wadler and colleagues also investigate the uniform treatment of renamings and substitutions in a simply typed setting [6] which originates from work by McBride [21]. This approach unifies repeated similar lemmas, such as merging all four compositionality lemmas into one. They hide the structural dependence of renamings and substitutions in a uniform, Mode-indexed vector (where V is the index for renamings and T is the index for substitutions) and use termination checking directly to make Agda see the structural dependence when it matters.

Saffrich [23] adopts a different approach and hides the structural dependence behind instance resolution. He works in a scoped syntax setting and represent the uniform structure for renamings and substitutions as functions, similar to the setting in this work. On top he provides reflection-based code generation for the syntax-dependent σ-calculus lemmas, which could possibly be adopted to our setting, again to avoid external code generation.

Adapting our approach with safe rewrite rules to a setting where renamings and substitutions can be treated uniformly would be a logical next step. However, for this to be useful in practice, the equational theory has to be extended in non-obvious ways, to support uniform proofs over renamings and substitutions. Furthermore, in this case we need a two-level rewrite system, one on the type level for normalizing expressions in the lattice of Modes, generated by $\mathsf{V} \sqsubseteq \mathsf{T}$ [6], and one for the adapted σ-calculus. When experimenting with this

approach, we found that the interaction of the two layers makes preservation of confluence difficult.

6 Conclusion

We have shown how to use the normalization procedure of the σ-calculus to generate definitional equalities in Agda. To this end, we closely followed the underlying equational theory of Autosubst 2, which provides the normalization procedure for Rocq to produce propositional equalities. We discovered that, to rewrite the full equational theory and maintain logical consistency, we want two contrary things: the full computational behavior by a modeling substitutions as functions, to instantiate of the equational theory with proofs, but also symbolic behavior to make the rules eligible for rewrite rules and higher-order pattern unification.

References

1. M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '90, page 31–46, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/96709.96712.
2. Andreas Abel and Brigitte Pientka. Higher-order dynamic pattern unification for dependent types and records. In Luke Ong, editor, *Typed Lambda Calculi and Applications*, pages 10--26, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
3. Robin Adams. Formalized metatheory with terms represented by an indexed family of types. In *Proceedings of the 2004 International Conference on Types for Proofs and Programs*, TYPES'04, page 1–16, Berlin, Heidelberg, 2004. Springer-Verlag. doi:10.1007/11617990_1.
4. Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type and scope safe universe of syntaxes with binding: Their semantics and proofs, 2021. URL: https://arxiv.org/abs/2001.11001, arXiv:2001.11001.
5. Guillaume Allais, James Chapman, Conor McBride, and James McKinna. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, CPP 2017, page 195–207, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3018610.3018613.
6. Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. Substitution without copy and paste. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)*, Birmingham, UK, 2025.
7. Thorsten Altenkirch and Ambrus Kaposi. Type theory in type theory using quotient inductive types. *SIGPLAN Not.*, 51(1):18–29, January 2016. doi:10.1145/2914770.2837638.
8. Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. Dedukti: a logical framework based on the $\lambda\pi$-calculus modulo theory, 2023. URL: https://arxiv.org/abs/2311.07185, arXiv:2311.07185.
9. Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. Strongly typed term representations in coq. *Journal of Automated Reasoning - JAR*, 49:1--19, 08 2012. doi:10.1007/s10817-011-9219-0.
10. Jesper Cockx. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In Marc Bezem and Assia Mahboubi, editors, *25th International Conference on Types for Proofs and Programs (TYPES 2019)*, volume 175 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1--2:27, Dagstuhl, Germany, 2020. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2019.2, doi:10.4230/LIPIcs.TYPES.2019.2.

11 Jesper Cockx and Andreas Abel. Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi:10.1145/3236770.

12 Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.*, 5(POPL), January 2021. doi:10.1145/3434341.

13 Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM*, 43(2):362–397, March 1996. doi:10.1145/226643.226675.

14 Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Phd thesis, Éditeur inconnu, 1972. URL: https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf.

15 Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. Controlling unfolding in type theory. *arXiv preprint arXiv:2210.05420*, 2022. URL: https://arxiv.org/abs/2210.05420.

16 Martin Hofmann. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory*, pages 13--54. Springer, 1997.

17 Jieh Hsiang. Refutational theorem proving using term-rewriting systems. *Artificial Intelligence*, 25(3):255--300, 1985. URL: https://www.sciencedirect.com/science/article/pii/0004370285900748, doi:10.1016/0004-3702(85)90074-8.

18 Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1--26:18, Dagstuhl, Germany, 2024. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2024.26, doi:10.4230/LIPIcs.ITP.2024.26.

19 Yann Leray and Théo Winterhalter. Encode the cake and eat it too: Controlling computation in type theory, locally. *Proc. ACM Program. Lang.*, 10(POPL), January 2026. doi:10.1145/3776704.

20 Per Martin-Löf. An intuitionistic theory of types: Predicative part. In H.E. Rose and J.C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 73--118. Elsevier, 1975. URL: https://www.sciencedirect.com/science/article/pii/S0049237X08719451, doi:10.1016/S0049-237X(08)71945-1.

21 Conor McBride. Type-preserving renaming and substitution. Unpublished manuscript, 2005. URL: http://strictlypositive.org/ren-sub.pdf.

22 John C. Reynolds. Towards a theory of type structure. In B. Robinet, editor, *Programming Symposium*, pages 408--425, Berlin, Heidelberg, 1974. Springer Berlin Heidelberg. URL: https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf.

23 Hannes Saffrich. Abstractions for Multi-Sorted Substitutions. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 32:1--32:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2024.32, doi:10.4230/LIPIcs.ITP.2024.32.

24 Hannes Saffrich, Peter Thiemann, and Marius Weidner. Intrinsically typed syntax, a logical relation, and the scourge of the transfer lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development*, TyDe 2024, page 2–15, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3678000.3678201.

25 Steven Schäfer, Gert Smolka, and Tobias Tebbi. Completeness and decidability of de bruijn substitution algebra in coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs*, CPP '15, page 67–73, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2676724.2693163.

26 Steven Schäfer, Tobias Tebbi, and Gert Smolka. Autosubst: Reasoning with de bruijn terms and parallel substitutions. In Christian Urban and Xingyuan Zhang, editors, *Interactive Theorem Proving*, pages 359--374, Cham, 2015. Springer International Publishing. URL: https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf.

27 Jeremy G. Siek and Tianyu Chen. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming*, 31:e30, 2021. doi:10.1017/S0956796821000241.

28 Kathrin Stark. *Mechanising Syntax with Binders in Coq*. PhD thesis, Saarland University, Saarbrücken, Germany, January 2020. Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.). URL: https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf.

29 Kathrin Stark, Steven Schäfer, and Jonas Kaiser. Autosubst 2: reasoning with multi-sorted de bruijn terms and vector substitutions. CPP 2019, page 166–180, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3293880.3294101.

30 Philip Wadler. Explicit weakening. *Electronic Proceedings in Theoretical Computer Science*, 413:15–26, November 2024. URL: http://dx.doi.org/10.4204/EPTCS.413.2, doi:10.4204/eptcs.413.2.